P3953R2
Rename `std::runtime_format`

Published Proposal, 2026-03-24

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

> There are only two hard things in Computer Science: cache invalidation and naming things.
>
> — Phil Karlton

1. Abstract

[P2918] introduced `std::runtime_format` to allow opting out of compile-time format string checks in `std::format`. Subsequently, [P3391] made `std::format` usable in constant evaluation. As a result, `std::runtime_format` can now be evaluated at compile time, making its name misleading. This paper proposes renaming `std::runtime_format` to `std::dynamic_format` to better reflect its semantics and avoid confusion in `constexpr` contexts.

2. Changes since R1

- Added wording.

3. Changes since R0

- Used the actual proposed name in the "after" example.

4. Motivation

The name `std::runtime_format` was accurate when introduced in [P2918], as format strings were not usable in constant evaluation. However, with the adoption of `constexpr` `std::format`, the term runtime no longer reliably describes the behavior of `std::runtime_format`.

Consider the following code:

```
constexpr auto f(std::string_view fmt, int value) {
  return std::format(std::runtime_format(fmt), value);
}
```

Despite its name, `std::runtime_format` can be evaluated at compile time. This creates a semantic mismatch:

- "runtime" suggests evaluation timing
- The facility actually describes how the format string is obtained.

The real distinction is not when formatting occurs, but how the format string is provided and validated. The term runtime conflates it with evaluation time.

§ 5. Proposed Naming: `std::dynamic_format`

The proposed name `std::dynamic_format` reflects the actual semantics:

- The format string is dynamically provided
- The format string is not a compile-time constant
- The validation is deferred (but may still occur during constant evaluation)

This aligns with existing terminology `std::format` such as dynamic format specifiers (`check_dynamic_spec`).

Example with proposed name:

```
constexpr auto f(std::string_view fmt, int value) {
  return std::format(std::dynamic_format(fmt), value);
}
```

This reads naturally and avoids semantic contradiction.

§ 6. Impact on existing code

If this is adopted for C++26 there will be no impact on existing code since `std::runtime_format` is a C++26 feature.

§ 7. Wording

Modify "Header `<format>` synopsis" [format.syn]:

```
template<class charT> struct ~~runtime~~dynamic-format-string {    // expositi
private:
  basic_string_view<charT> str;    // exposition only
public:
  constexpr ~~runtime~~dynamic-format-string(basic_string_view<charT> s) noexce
  ~~runtime~~dynamic-format-string(const ~~runtime~~dynamic-format-string&) = dele
  ~~runtime~~dynamic-format-string& operator=(const ~~runtime~~dynamic-format-stri
};
constexpr ~~runtime~~dynamic-format-string<char>
```

```
  ~~runtime~~dynamic_format(string_view fmt) noexcept { return fmt; }
constexpr ~~runtime~~dynamic-format-string<wchar_t>
  ~~runtime~~dynamic_format(wstring_view fmt) noexcept { return fmt; }
```

Modify "Class template `basic_format_string`" [format.fmt.string]:

```
namespace std {
  template<class charT, class... Args>
  struct basic_format_string {
  private:
    basic_string_view<charT> str;    // exposition only

  public:
    template<class T> consteval basic_format_string(const T& s);
    constexpr basic_format_string(~~runtime~~dynamic-format-string<charT> s) no

    constexpr basic_string_view<charT> get() const noexcept { return str; }
  };
}
```

§ References

§ Informative References

[P2918]

Victor Zverovich. *Runtime format strings II*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2918r2.html

[P3391]

Barry Revzin. `*constexpr std::format*`. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3391r2.html